

Plan del Proyecto

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso: Aseguramiento de la Calidad del Software

Periodo: I Semestre 2026

Estudiantes:

Angie Aguilar Alemán

Jonathan Sancho Loaiza

Jeremy Montero Araya

Dereck Cuadra Gonzalez

Docente: Prof. Marcos Rodriguez



Tabla de Contenidos

Índice de Tablas	3
Introducción	4
Especificaciones del software	5
Versiones y proveedores de la aplicación	5
Hardware utilizado en el entorno de ejecución de la aplicación	6
Descripción de requerimientos o historias de usuario	6
Anexo Vocabulario Técnico Descripción de requerimientos o historias de usuario	6
Problema	18
Situación actual del sistema	18
Descripción del problema	19
Origen del problema	19
Consecuencias identificadas	20
Objetivos, metas e indicadores	20
Objetivo General	20
Objetivos Específicos	20
Metas	21
Indicadores	21
Recursos Disponibles	21
Infraestructura y entorno de trabajo	22
Datos y cuentas de prueba	22
Herramientas para la ejecución de pruebas	22
Código fuente y documentación disponible	22
Cronograma de actividades	23
Bibliografía	26

Índice de Tablas

Tabla 1: Versiones y proveedores de la aplicación	5
Tabla 2: Hardware utilizado en el entorno de ejecución de la aplicación	6
Tabla 3: Vocabulario Técnico	7
Tabla 4: Historia de Usuario 1	7
Tabla 5: Historia de Usuario 2	9
Tabla 6: Historia de Usuario 3	10
Tabla 7: Historia de Usuario 4	12
Tabla 8: Historia de Usuario 5	13
Tabla 9: Historia de Usuario 6	15
Tabla 10: Historia de Usuario 7	16
Tabla 11: Historia de Usuario 8	17
Tabla 12: Recursos disponibles	22
Tabla 13: Cronograma de actividades	22

Introducción

Hoy en día, muchas empresas y organizaciones necesitan sistemas digitales para gestionar sus operaciones internas, como el registro de pedidos, el control de usuarios y la protección de la información que manejan. Cuando estos sistemas no funcionan correctamente o no cuentan con las medidas de seguridad adecuadas, pueden ocurrir problemas graves: desde la pérdida de información importante hasta el acceso no autorizado de personas ajenas al sistema.

En este contexto, se identificó una aplicación web de gestión de pedidos que permite a los usuarios registrarse, iniciar sesión y crear pedidos de forma digital. El sistema cuenta con dos tipos de usuarios: los usuarios regulares, que pueden gestionar sus propios pedidos, y los administradores, que tienen acceso completo para supervisar todos los pedidos y usuarios registrados. Para proteger el acceso, el sistema utiliza un mecanismo de autenticación basado en tokens, lo que significa que cada vez que un usuario inicia sesión, el sistema le genera una credencial temporal que le permite operar de forma segura.

Se seleccionó este proyecto porque, a pesar de que el sistema se encuentra funcional, se detectaron áreas de oportunidad relacionadas con la calidad del software y la seguridad de la información. Al tratarse de un sistema que maneja datos de usuarios y controla el acceso mediante roles y credenciales, cualquier falla o vulnerabilidad podría comprometer tanto la operación como la confianza de quienes lo utilizan. Por esta razón, resulta un caso de estudio ideal para aplicar prácticas de aseguramiento de calidad, realizar pruebas estructuradas y proponer mejoras concretas.

El objetivo de este documento es presentar de manera clara el plan de trabajo que guiará las actividades del proyecto. En él se describen las características técnicas del sistema, las funcionalidades que serán evaluadas, el problema central identificado junto con sus causas y efectos, así como los objetivos, metas e indicadores que permitirán medir el avance y el impacto de las mejoras realizadas.

Dado que el sistema involucra el manejo de credenciales, tokens de sesión y distintos niveles de acceso, resulta especialmente relevante aplicar técnicas de verificación que abarquen tanto el comportamiento funcional como los aspectos de seguridad. Un enfoque de pruebas bien definido no solo permite detectar errores antes de que generen consecuencias, sino que también establece un marco de referencia para futuras mejoras sobre la base de código existente.

A lo largo del documento se incluyen también los recursos con los que cuenta el equipo para llevar a cabo las pruebas, la planificación temporal de las actividades y las fuentes consultadas para sustentar las decisiones tomadas durante el proyecto.

Especificaciones del software

El presente documento describe las tecnologías de software y el hardware utilizados en el entorno de ejecución de la aplicación. La aplicación se encuentra encapsulada por medio de contenedores Docker, lo que permite desplegar todos sus componentes (frontend, backend y base de datos) de forma aislada y reproducible dentro de un mismo equipo. En la tabla de versiones, la columna "Tecnología / Componente" indica el nombre de la herramienta o librería utilizada junto con una breve descripción de su función, mientras que la columna "Proveedor y Versión" especifica quién desarrolla dicha tecnología y la versión exacta empleada en el proyecto.

Versiones y proveedores de la aplicación

Tabla 1: Versiones y proveedores de la aplicación

Tecnología / Componente	Proveedor y Versión
Docker Plataforma de contenedores que encapsula la aplicación y sus dependencias en ambientes aislados y reproducibles.	Proveedor: Docker, Inc. Versión: Dependiente del entorno de ejecución
Node.js Entorno de ejecución de JavaScript del lado del servidor, utilizado para el desarrollo del frontend.	Proveedor: OpenJS Foundation Versión: 18.0.0
React Biblioteca de JavaScript para la construcción de interfaces de usuario basadas en componentes.	Proveedor: Meta Platforms, Inc. Versión: 18.3.1
React Router DOM Biblioteca de enrutamiento para aplicaciones React en el navegador.	Proveedor: Remix Software Versión: 7.12.0
Axios Cliente HTTP basado en promesas para realizar peticiones desde el frontend hacia el backend.	Proveedor: Axios Project Versión: 1.13.12
Java Lenguaje de programación orientado a objetos utilizado para el desarrollo del backend.	Proveedor: Oracle Corporation Versión: 25.0.2
Spring Boot Framework de Java que simplifica la creación de aplicaciones backend con configuración mínima.	Proveedor: VMware / Broadcom (Spring) Versión: 4.0.1
JSON Web Token (JWT) Biblioteca para la generación y validación de tokens JWT, utilizada en la autenticación.	Proveedor: JWT Project Versión: 0.13.0

SpringDoc OpenAPI Herramienta que genera documentación interactiva de la API REST de forma automática.	Proveedor: SpringDoc Versión: 3.0.1
PostgreSQL Sistema gestor de bases de datos relacional. Se ejecuta como imagen Docker oficial (postgres:18.0.0).	Proveedor: PostgreSQL Global Development Group Versión: 18.0.0

Hardware utilizado en el entorno de ejecución de la aplicación

Todo el software descrito en este documento (frontend, backend y base de datos) se ejecuta en un solo equipo, desplegado mediante contenedores Docker. A continuación, se detallan las especificaciones del equipo utilizado.

Tabla 2: Hardware utilizado en el entorno de ejecución de la aplicación

Componente	Especificación
Equipo	ASUS TUF Gaming A15 FA506NC
Procesador	AMD Ryzen 5 7535HS
Memoria RAM	32 GB DDR5 — 4800 MHz
Tarjeta gráfica	NVIDIA GeForce RTX 3050 Mobile
Almacenamiento (1)	NVMe SSD — Patriot M.2 P300 — 512 GB
Almacenamiento (2)	NVMe SSD — Micron MTFDKBA512QGN — 512 GB
Red cableada	Realtek RTL8111 — Gigabit Ethernet
Red inalámbrica	Realtek RTL8852BE — IEEE 802.11ax (Wi-Fi 6)

Descripción de requerimientos o historias de usuario

Anexo Vocabulario Técnico Descripción de requerimientos o historias de usuario

Tabla 3: Vocabulario Técnico

Término	Descripción
Formato ISO 8601	Estándar para fechas y horas: YYYY-MM-DDTHH:MM:SS.ss+HH:MM. Ej: 2020-05-07T23:48:09.06+01:00
ID (Identificador Único)	Número entero positivo único generado automáticamente por el sistema
Texto Alfanumérico	Cadena con letras (A-Z, a-z) y números (0-9), puede permitir espacios
Validación de Campos	Verificación de datos: longitud, formato y obligatoriedad
Autenticación	Proceso de verificación de identidad mediante usuario y contraseña
Dashboard	Pantalla principal con métricas como total de usuarios y órdenes
CRUD	Operaciones: Crear, Leer, Actualizar, Eliminar
Coincidencia Parcial	Búsqueda que coincide aunque el texto no sea exacto
Case Insensitive	No diferencia entre mayúsculas y minúsculas
Base de Datos	Sistema de almacenamiento persistente de datos
Mensajes de Error	Respuestas ante fallos: específicos o generales
Tiempo Real / Datos Consistentes	Datos actuales sin retrasos ni inconsistencias

Tabla 4: Historia de Usuario 1

US-AUTH-001 · Registro de usuario

Historia de Usuario	Como usuario nuevo quiero registrarme en el sistema para poder acceder a la plataforma y utilizar sus funcionalidades.
Criterios de Aceptación	
Escenario 1: Registro exitoso	
Given	El usuario se encuentra en el formulario de registro que contiene los campos obligatorios: usuario, contraseña, nombre y correo electrónico
When	Ingresa sus datos en los campos requeridos con el siguiente formato: • Usuario: texto alfanumérico, sin espacios, mínimo 4 caracteres, máximo 20, único • Contraseña: mínimo 8 caracteres, al menos una letra y un número • Nombre: texto alfabético, permite espacios, mínimo 2 caracteres • Correo electrónico: formato válido tipo texto@dominio.com y presiona "Registrarse"
Then	El sistema crea la cuenta y redirige al usuario al dashboard
Escenario 2: Datos incompletos	
Given	El usuario se encuentra en el formulario de registro que contiene los campos obligatorios: usuario, contraseña, nombre y correo electrónico
When	Intenta registrarse dejando uno o más de los siguientes campos vacíos: • Usuario • Contraseña • Nombre • Correo electrónico

Then	El sistema no permite el registro y muestra mensajes de error específicos por cada campo vacío: • Si el campo usuario está vacío: "El usuario es requerido" • Si el campo contraseña está vacío: "La contraseña es requerida" • Si el campo nombre está vacío: "El nombre es requerido" • Si el campo correo electrónico está vacío: "El correo electrónico es requerido" Además, el sistema muestra un mensaje general indicando la cantidad de campos faltantes y cuáles son, por ejemplo: • "Faltan 2 campos por completar: usuario y contraseña" • "Faltan 4 campos por completar: usuario, contraseña, nombre y correo electrónico"
------	--

Tabla 5: Historia de Usuario 2

US-AUTH-002 · Inicio de sesión	
Historia de Usuario	Como usuario registrado quiero iniciar sesión en el sistema para acceder a mi cuenta y a las funcionalidades disponibles.
Criterios de Aceptación	
Escenario 1: Inicio de sesión exitoso	
Given	El usuario se encuentra en la página de inicio de sesión que contiene los campos: usuario y contraseña
When	Ingresa sus credenciales con el siguiente formato: • Usuario: texto alfanumérico, sin espacios mínimo 4 y máximo 20 caracteres

	Contraseña: texto con mínimo 8 caracteres al menos una letra y un número y presiona el botón "Iniciar sesión"
And	Las credenciales coinciden con un usuario registrado en el sistema
Then	El sistema lo redirige al dashboard
Escenario 2: Credenciales incorrectas	
Given	El usuario se encuentra en la página de inicio de sesión que contiene los campos: usuario y contraseña
When	Ingresa sus credenciales en los campos requeridos con el siguiente formato: - Usuario: texto alfanumérico, sin espacios mínimo 4 y máximo 20 caracteres - Contraseña: texto con mínimo 8 caracteres al menos una letra y un número y presiona el botón "Iniciar sesión"
Then	El sistema no permite el acceso y muestra el siguiente mensaje de error: "Usuario o contraseña incorrectos"

Tabla 6: Historia de Usuario 3

US-DASH-003 · Visualización del Dashboard	
Historia de Usuario	Como usuario autenticado quiero visualizar el dashboard del sistema para conocer la cantidad de usuarios y órdenes registradas.
Criterios de Aceptación	
Escenario 1: Mostrar estadísticas	

Given	El usuario ha iniciado sesión correctamente y se encuentra en el dashboard y existen datos registrados en el sistema: • Usuarios: uno o más registros válidos en la base de datos • Órdenes: uno o más registros válidos en la base de datos
When	Accede al dashboard
Then	El sistema muestra una tarjeta con el total de usuarios registrados con el siguiente formato: • Valor numérico entero positivo (0 o mayor) • Sin decimales • Representa la cantidad total de usuarios almacenados en el sistema
And	El sistema muestra una tarjeta con el total de órdenes registradas con el siguiente formato: • Valor numérico entero positivo (0 o mayor) • Sin decimales • Representa la cantidad total de órdenes almacenadas en el sistema
And	Los valores mostrados corresponden exactamente a los datos actuales almacenados en el sistema (sin desfase, datos en tiempo real o última actualización consistente)

Tabla 7: Historia de Usuario 4

Historia de Usuario	Como usuario autenticado quiero crear una nueva orden para registrar información dentro del sistema.
Criterios de Aceptación	
Escenario 1: Creación exitosa de orden	
Given	El usuario ha iniciado sesión correctamente y se encuentra en la sección de órdenes(apartado llamado userPage en el header de la página) y visualiza el formulario de creación de orden que contiene el campo obligatorio: descripción
When	Ingresa la descripción de la orden con el siguiente formato: • Descripción: texto alfanumérico • Permite espacios • Mínimo 5 caracteres • Máximo 255 caracteres
And	Presiona el botón de guardar
Then	El sistema valida los datos ingresados y registra la nueva orden correctamente en el sistema con los siguientes datos: • ID: valor único, entero positivo, generado automáticamente • Fecha y hora: timestamp generado automáticamente en formato ISO 8601 (YYYY-MM-DDTHH:MM:SS.ss+HH:MM, por ejemplo: 2020-05-07T23:48:09.06+01:00) • Descripción: texto ingresado por el usuario
And	La orden queda almacenada con los datos generados y es visible en el listado de órdenes mostrando: • ID como número entero • Fecha y hora en formato ISO 8601 • Descripción como texto

Escenario 2: Campos vacíos	
Given	El usuario se encuentra en el formulario de creación de orden que contiene el campo obligatorio: descripción
When	Deja el siguiente campo vacío: • Descripción
And	Presiona el botón "Guardar"
Then	El sistema muestra un mensaje de error
And	El sistema muestra un mensaje general indicando los campos faltantes, por ejemplo: • "Falta 1 campo por completar: descripción"

Tabla 8: Historia de Usuario 5

US-ORD-005 · Visualizar órdenes	
Historia de Usuario	Como usuario autenticado quiero visualizar las órdenes registradas para consultar la información existente en el sistema.
Criterios de Aceptación	
Escenario 1: Mostrar lista de órdenes	
Given	El usuario ha iniciado sesión correctamente y existen órdenes registradas en el sistema
When	El usuario accede a la sección de órdenes (apartado llamado userPage en el header de la página)
Then	El sistema muestra la lista de órdenes existentes con los siguientes datos por cada registro: • ID: ○ Valor único

	○ Número entero positivo ○ Generado automáticamente ● Fecha y hora: ○ Formato ISO 8601 ○ Estructura: YYYY-MM-DDTHH:MM:SS.ss+HH:MM ○ Ejemplo: 2020-05-07T23:48:09.06+01:00 ● Descripción: ○ Texto alfanumérico ○ Permite espacios ○ Entre 5 y 255 caracteres
And	Cada orden se muestra en un elemento de lista o fila de tabla
And	La lista contiene todas las órdenes registradas en el sistema sin omitir registros
And	Los datos mostrados corresponden exactamente a la información almacenada en el sistema

Tabla 9: Historia de Usuario 6

US-ADM-006 · Buscar usuarios	
Historia de Usuario	Como administrador quiero buscar usuarios por nombre para encontrar rápidamente un usuario en el sistema.
Criterios de Aceptación	
Escenario 1: Búsqueda exitosa	
Given	El administrador ha iniciado sesión correctamente y se encuentra en el panel de gestión de usuarios(apartado llamado en adminPage ubicado en el header, seleccionando el apartado de users)

	y existen usuarios registrados en el sistema con los siguientes datos: • Usuario: texto alfanumérico, sin espacios, único, entre 4 y 20 caracteres • Contraseña: mínimo 8 caracteres, al menos una letra y un número (almacenada de forma segura) • Nombre: texto alfabético, permite espacios, mínimo 2 caracteres • Correo electrónico: formato válido tipo texto@dominio.com
When	Ingresa un valor en el buscador con el siguiente formato: • Nombre: texto alfabético • Permite espacios • Mínimo 1 carácter
Then	El sistema filtra y muestra los usuarios que coinciden con la búsqueda con las siguientes reglas: • Coincidencia parcial o completa sobre el campo nombre • No distingue entre mayúsculas y minúsculas
And	El sistema muestra los resultados en formato de lista o tabla incluyendo los siguientes datos por usuario: • Usuario: texto alfanumérico • Nombre: texto • Correo electrónico: formato válido
And	La contraseña no se muestra en los resultados por motivos de seguridad
And	Los resultados corresponden exactamente a los datos almacenados en el sistema sin omitir coincidencias válidas

Tabla 10: Historia de Usuario 7

US-ADM-007 · Eliminar usuario	
Historia de Usuario	Como administrador quiero eliminar usuarios del sistema para mantener la base de datos actualizada.
Criterios de Aceptación	
Escenario 1: Eliminación de usuario	
Given	El administrador ha iniciado sesión correctamente y se encuentra en el panel de gestión de usuarios y visualiza la lista de usuarios registrados con los siguientes datos por cada usuario: • Usuario: texto alfanumérico, sin espacios, único, entre 4 y 20 caracteres • Contraseña: mínimo 8 caracteres, al menos una letra y un número (almacenada de forma segura, no visible) • Nombre: texto alfabético, permite espacios, mínimo 2 caracteres • Correo electrónico: formato válido tipo texto@gmail.com
When	Selecciona la opción "Eliminar" sobre un usuario específico identificado por: • Usuario único (texto alfanumérico)
Then	El sistema elimina el usuario del sistema
And	El usuario eliminado ya no se muestra en la lista de usuarios
And	Los cambios se reflejan inmediatamente en la interfaz mostrando la lista actualizada sin el usuario eliminado

Tabla 11: Historia de Usuario 8

US-ADM-008 · Buscar órdenes	
Historia de Usuario	Como administrador quiero buscar órdenes por descripción para localizar órdenes específicas en el sistema.
Criterios de Aceptación	
Escenario 1: Búsqueda de orden	
Given	El administrador ha iniciado sesión correctamente y se encuentra en la gestión de órdenes(apartado llamado en adminPage ubicado en el header, seleccionando el apartado de orders) y existen órdenes registradas en el sistema con los siguientes datos: • ID: valor único, entero positivo • Fecha y hora: formato ISO 8601 (YYYY-MM-DDTHH:MM:SS.ss+HH:MM) • Descripción: texto alfanumérico, permite espacios, entre 5 y 255 caracteres
When	Ingresa un valor en el buscador con el siguiente formato: • Descripción: texto alfanumérico • Permite espacios • Mínimo 1 carácter
Then	El sistema filtra y muestra las órdenes que coinciden con la búsqueda con las siguientes reglas: • Coincidencia parcial o completa sobre el campo descripción • No distingue entre mayúsculas y minúsculas

And	El sistema muestra los resultados en formato de lista o tabla incluyendo los siguientes datos por orden: • ID: número entero positivo • Fecha y hora: formato ISO 8601 • Descripción: texto
And	Los resultados corresponden exactamente a los datos almacenados en el sistema sin omitir coincidencias válidas

Problema

Situación actual del sistema

La aplicación evaluada es un sistema web de gestión de pedidos construido con Spring Boot en el backend y React en el frontend, con PostgreSQL como motor de base de datos. El sistema distingue entre dos perfiles de acceso: usuarios regulares, que pueden crear y consultar sus pedidos, y administradores, que disponen de permisos ampliados para gestionar usuarios y órdenes de todo el sistema. La autenticación se resuelve mediante JSON Web Tokens (JWT), generados al momento del inicio de sesión y almacenados en el navegador del cliente. Aunque la aplicación cumple con su propósito funcional básico, una inspección inicial del código y de su configuración reveló debilidades importantes en materia de calidad y seguridad. No se encontraron suites de prueba activas ni en el backend ni en el frontend, y varias decisiones de configuración presentan riesgos que podrían pasar inadvertidos sin un proceso formal de revisión.

Descripción del problema

El proyecto carece de un mecanismo de verificación que permita confirmar, de forma repetible y documentada, que las funcionalidades del sistema operan según lo esperado y que las configuraciones de seguridad son adecuadas. Esta ausencia de control de calidad abarca tanto la capa de servidor como la de cliente, y se extiende a los archivos de configuración del entorno de despliegue.

Origen del problema

A partir del análisis exploratorio del repositorio, se identificaron las siguientes situaciones que dan origen al problema:

1. El backend no cuenta con pruebas unitarias ni de integración. Las dependencias de testing (JUnit, Mockito) están declaradas en el proyecto pero no se utilizan en ningún caso de prueba funcional.
2. No existen pruebas que validen la comunicación entre el frontend en React y la API REST del backend, dejando sin cobertura los flujos completos del usuario.
3. Las credenciales de conexión a PostgreSQL aparecen escritas directamente en los archivos de propiedades del servidor, sin hacer uso de variables de entorno o mecanismos de gestión de secretos.
4. El archivo docker-compose emplea contraseñas triviales e idénticas al nombre de usuario para el servicio de base de datos.
5. El servidor acepta peticiones desde cualquier origen, con cualquier método y cabecera HTTP, debido a una política CORS sin restricciones.
6. La comprobación de vigencia del token JWT ocurre exclusivamente en código JavaScript del cliente, sin que el servidor valide por su cuenta si el token ha expirado. Además, el token se guarda en localStorage, exponiéndolo a posibles ataques de tipo XSS.

Consecuencias identificadas

De no abordarse las situaciones descritas, el sistema queda expuesto a los siguientes escenarios:

7. Errores en flujos clave (registro, login, creación de pedidos) podrían llegar a producción sin ser detectados, afectando directamente la experiencia de quienes usan la plataforma.
8. Un atacante podría aprovechar la política CORS abierta o la validación deficiente de tokens para realizar peticiones no autorizadas a la API, accediendo o modificando información restringida.
9. Las credenciales visibles en el repositorio representan un punto de entrada directo a la base de datos si el código fuente quedara expuesto, comprometiendo todos los registros almacenados.
10. La corrección de defectos descubiertos en etapas avanzadas del desarrollo implica un esfuerzo considerablemente mayor que si se detectaran mediante pruebas tempranas y sistemáticas.
11. Sin métricas de calidad definidas, no existe forma objetiva de determinar en qué estado se encuentra el software ni de medir el progreso de las mejoras aplicadas.

Ante este panorama, el presente proyecto se propone implementar un proceso de aseguramiento de calidad que cubra los aspectos funcionales y de seguridad del sistema, permitiendo documentar hallazgos, corregir deficiencias y establecer una base sólida para el mantenimiento futuro de la aplicación.

Objetivos, metas e indicadores

Objetivo General

Comprobar la calidad del software mediante la implementación de estándares técnicos y metodologías de aseguramiento de la calidad reconocidas por la industria.

Objetivos Específicos

1. Analizar los requerimientos del sistema mediante la revisión y validación de historias de usuario y sus criterios de aceptación, con el fin de establecer una base adecuada para el proceso de pruebas.
2. Diseñar y estructurar los casos de prueba que permitan validar los flujos funcionales del sistema, asegurando la cobertura de los escenarios principales.
3. Ejecutar pruebas manuales y automatizar los casos de prueba prioritarios del sistema, aplicando técnicas de aseguramiento de la calidad para detectar defectos y evaluar el comportamiento del software.
4. Evaluar los resultados del proceso de pruebas mediante el análisis de defectos y la generación de informes, con el propósito de determinar el nivel de calidad del software.

Metas

1. Documentar y validar al menos el 80% de las historias de usuario antes del cierre del plan del proyecto.
2. Definir al menos 60 casos de prueba documentados antes de la entrega del plan de pruebas.
3. Ejecutar al menos el 80% de los casos de prueba definidos y automatizar al menos el 50% de los casos de prueba críticos, registrando evidencia de ejecución.
4. Generar un informe final de pruebas que incluya defectos clasificados, métricas y conclusiones sobre la calidad del sistema.

Indicadores

1. Porcentaje de historias validadas $\geq 80\%$, revisadas y aprobadas por el equipo
2. Número de casos de prueba diseñados ≥ 60 , documentados y entregados en el plan de pruebas.
3. Porcentaje de ejecución $\geq 80\%$ y porcentaje de automatización $\geq 50\%$, con evidencia documentada y scripts funcionales en el repositorio.
4. Informe de pruebas entregado, que incluya clasificación de defectos por severidad, métricas y conclusiones.

Recursos Disponibles

Para llevar a cabo las actividades de prueba sobre el sistema, el equipo cuenta con los siguientes recursos técnicos y de información:

Tabla 12: Recursos disponibles

Categoría	Recurso	Descripción
Infraestructura y entorno	Docker Compose	Permite ejecutar backend (Spring Boot), frontend (React) y base de datos (PostgreSQL) en contenedores.
	Entorno local	Cada integrante trabaja en un entorno idéntico, evitando problemas de compatibilidad.
Datos y cuentas de prueba	Base de datos de prueba	Contiene datos ficticios que simulan escenarios reales (usuarios, pedidos, estados).
	Cuentas de prueba	Usuarios predefinidos con distintos roles (administrador y usuario).
Herramientas de pruebas	JUnit 5 y Mockito	Pruebas unitarias e integración en backend.
	Swagger (SpringDoc OpenAPI)	Visualización y prueba de endpoints de la API.
	Postman	Ejecución de solicitudes y validación de escenarios.
	Testing en React	Validación del comportamiento de componentes frontend.
Código y documentación	Repositorio del proyecto	Acceso completo al código fuente y configuración del sistema.

	Documentación API	Endpoints, parámetros y respuestas disponibles para diseño de pruebas.
--	-------------------	--

Cronograma de actividades

El siguiente cronograma organiza temporalmente las actividades planificadas a lo largo del semestre. Cada semana incluye las tareas a realizar, los responsables y los entregables esperados.

Tabla 13: Cronograma de actividades

Periodo	Tareas principales	Asignación	Entregable
Sem. 4 (9-15 mar)	Exploración inicial del repositorio y puesta en marcha del entorno local con Docker. Conformación del equipo y asignación de roles. Primera identificación de problemas mediante revisión del código.	Todos	Entorno funcional, equipo conformado.
Sem. 5 (16-22 mar)	Redacción de especificaciones técnicas (software y hardware). Escritura de las historias de usuario con sus criterios de aceptación. Identificación de	Todos	Borrador de secciones del plan.

	recursos para pruebas.		
Sem. 6 (23-29 mar)	Cierre del documento Plan del Proyecto: completar problema ampliado, cronograma, bibliografía. Revisión cruzada entre integrantes.	Todos	Plan del Proyecto entregado (E1).
Sem. 7 (30 mar - 5 abr)	Incorporar observaciones del profesor sobre el plan. Definir la estrategia general de pruebas: niveles, tipos y prioridades.	Todos	Plan ajustado según retroalimentación.
Sem. 8 (6-12 abr)	Diseño detallado de los casos de prueba para los flujos de autenticación y gestión de usuarios. Estructurar el documento del plan de pruebas.	Angie, Dereck	Casos de prueba de auth y usuarios.
Sem. 9 (13-19 abr)	Diseño de casos de prueba para gestión de órdenes, dashboard y seguridad. Seguimiento con el profesor. Preparar datos de prueba en la BD.	Jonathan, Jeremy	Casos restantes. Reunión de avance.

Sem. 10 (20-26 abr)	Consolidar el Plan de Pruebas: integrar todos los casos, redactar conclusiones y revisar coherencia general del documento.	Todos	Plan de Pruebas finalizado.
Sem. 11 (27 abr - 3 may)	Entrega formal del Plan de Pruebas. Correcciones menores de última hora si son necesarias.	Todos	Plan de Pruebas entregado (E2).
Sem. 12 (4-10 may)	Configurar JUnit y Mockito en el backend. Escribir las primeras pruebas automatizadas sobre endpoints de autenticación y filtros de seguridad.	Jonathan, Jeremy	Primeros scripts funcionales.
Sem. 13 (11-17 may)	Ampliar cobertura: controladores de órdenes, validaciones de token, escenarios negativos. Registrar defectos encontrados.	Jonathan, Jeremy	Cobertura $\geq$ 80% de casos.
Sem. 14 (18-24 may)	Completar automatización pendiente. Ejecutar suite completa y verificar	Todos	Suite completa ejecutada.

	resultados. Ajustar pruebas que fallen.		
Sem. 15 (25-31 may)	Entrega del Proyecto de Pruebas. Comenzar informe: compilar resultados, generar tablas y gráficos de cobertura.	Todos	Proyecto de Pruebas (E3).
Sem. 16 (1-7 jun)	Redactar informe final: hallazgos, clasificación de defectos, conclusiones y reflexiones del equipo. Preparar presentación.	Angie, Dereck	Informe de pruebas completo.
Sem. 17 (8-14 jun)	Entrega del informe, presentación oral del proyecto y diario reflexivo.	Todos	Informe, presentación, diario (E4, E5, E6).

Bibliografía

- [1] R. S. Pressman y B. R. Maxim, Ingeniería del Software: Un Enfoque Práctico, 8.^a ed. McGraw-Hill, 2015.
- [2] ISTQB, "Certified Tester Foundation Level Syllabus," International Software Testing Qualifications Board, versión 4.0, 2023. [En línea]. Disponible en: <https://www.istqb.org/certifications/certified-tester-foundation-level>.
- [3] OWASP Foundation, "OWASP Top Ten," 2021. [En línea]. Disponible en: <https://owasp.org/www-project-top-ten/>.
- [4] Auth0, "Introduction to JSON Web Tokens," 2024. [En línea]. Disponible en: <https://jwt.io/introduction>.

[5] Spring, "Spring Security Reference," VMware / Broadcom, 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/index.html>.